Aplicando ViewModel al Gestor de Colecciones

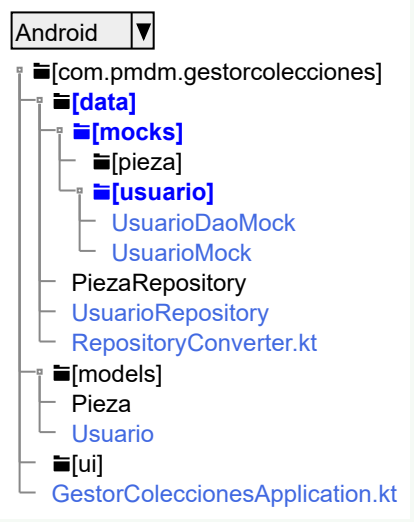
Descargar estos apuntes

Ejercicio

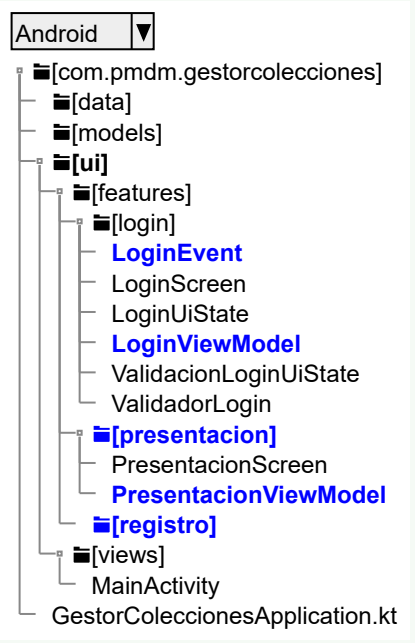
En el siguiente ejercicio partiremos del ejercicio 3 del Bloque 3.3 donde definíamos los elementos composables para crear el LoginScreen y el RegistroScreen. Ahora vamos a ir un paso más allá y añadir la clases necesarias para llevar la lógica de la aplicación a los distintos `ViewModel` además de completar la fuente de datos que teníamos en el ejercicio del que partimos. Para ello vamos a añadir toda la estructura de paquetes correspondientes a data y a model necesarios para seguir con la arquitectura que hemos propuesto en este curso.

Paquete Data y Model

Como ya hemos visto en ejercicios anteriores, debemos separar las fuentes de datos de la lógica de empresa, por lo que nos crearemos los paquetes necesarios **y que ya conocemos sobradamente**, con la data class **UsuarioMock** (piensa en los campos que te hacen falta para dar la funcionalidad que puedes ver en el screen), con la clase **UsuarioDaoMock** (en la que tendremos una lista de usuarios con algunos usuarios para pruebas), la clase para el repository y completar la clase para los mapeados de datos. Además, dentro del paquete models crearemos nuestra clase modelo Usuario con las mismas propiedades que UsuarioMock.



Paquete Ui



Dentro del paquete `ui.features.login` vamos a añadir y completar las clases necesarias para crear la lógica de negocia con la ayuda de ViewModel. Para ello completaremos la **interface sellada LoginEnvent** para que recoja y centralice todos los eventos que ocurran en la pantalla, por ejemplo:

- OnContraseñaCambiada
- OnRegistrarse
- OnIniciarSesionClick

Además crearemos la clase estrella de este ejercicio **LoginViewModel**, en la que tendremos la lógica de la aplicación y que nos hará de nexo entre las fuentes de datos y la interface de usuario. Recuerda que deberemos crear un objeto de tipo *UsuarioRepository* para acceder a la fuente de datos y poder usar las funciones *CRUD* de esta. Desde la parte de Login solo podremos introducir alguno de los usuarios que tengamos en los datos Mock, ya que al no tener la navegación disponible no podremos movernos entre pantallas.
En esta clase además, deberemos tener todas las variables **StateFul** necesarias, para seguir la lógica de **state-hoisting** (elevación de estado) vista en los temas. Y el método **onLoginEvent** para gestionar los eventos.

Una vez tengamos el paquete **login** completado con toda la funcionalidad, pasaremos a instanciar el ViewModel del login en la Main y a probar el funcionamiento de esta pantalla.

> **Tip**
>
> Para finalizar, tendremos que completar el paquete **registro** de la misma manera que el de login y crear la clase **PresentacionVieModel**, para llevar la lógica de la presentación a su ViewModel.

> **Warning**

Para mostrar los avisos de entrar a la aplicación si hemos introducido un login y password correcto o en caso incorrecto indicar que no existe el login o password. Podemos usar un layout `Box` que envuelva todo nuestro contenido y un texto alineado en la parte inferior que se muestre o no con el mensaje correspondiente, puede desaparecer cuando detecte algún cambio en los `TextField`.